

Prompt tuning(BNT问题) 调研总结

直接解决BNT问题的相关论文

1.DePT (CVPR2024)

DePT:单提示, 双特征头;

DPC: 双提示, 单特征流。

一个独特的观察视角:

$$\mathbf{CI}^{(r)} = \frac{1}{N} \sum_{j=1}^N \frac{\text{ReLU}(\bar{e}_*^{(r)} \bar{f}_j^{(r)})}{1/M \sum_{i=1}^M \text{ReLU}(\bar{e}_i^{(r)} \bar{f}_j^{(r)})}, \quad (3)$$

- **分子部分:** $\bar{f}_j^{(r)}$ 是第 j 张图片的特征在第 r 个通道的值; $\bar{e}_*^{(r)}$ 是它**真实对应文本 (Ground-truth) **在这个通道的值。分子计算的是: 在这个通道上, 图片和它正确标签的相似度有多高。 [🔗](#)
- **分母部分:** $\bar{e}_i^{(r)}$ 是所有 M 个类别的文本特征。分母计算的是: 在这个通道上, 图片和所有类别的平均相似度有多高。 [🔗+1](#)
- **ReLU 的作用:** 去除负值, 防止除以 0。 [🔗](#)
- **物理直觉:** 如果一个通道 r 能够让图片极其靠近它的正确类别 (分子大), 同时又远离其他所有错误类别 (分母小), 那么这个通道在做分类任务时就**非常重要**。这就是 CI 值的核心逻辑。 [🔗](#)

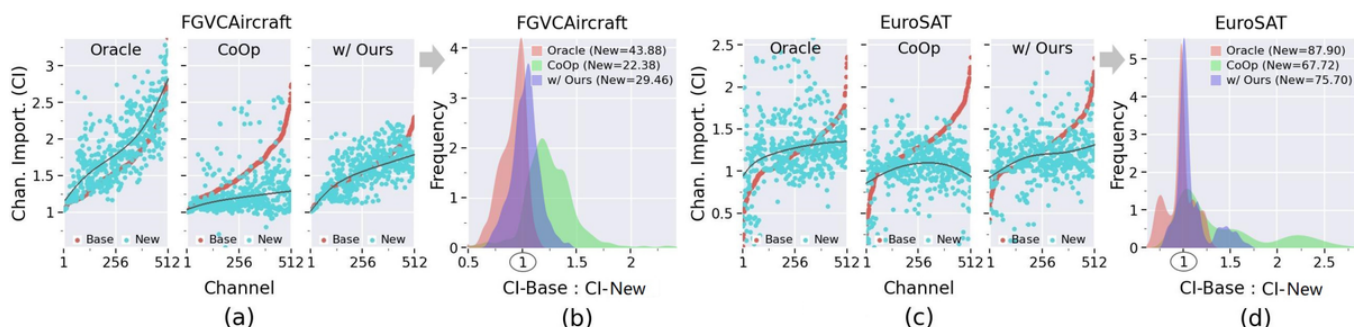


Figure 3. Channel Importance (CI) distributions of base and new tasks learned by the Oracle model and CoOp [58] w/ or w/o our DePT on the datasets FGVC Aircraft [32] and EuroSAT [14]. In (a)(c), the indexes of channels in the x -axis are reordered based on the CI of the base task, a blue/red point indicates a channel. In (b)(d), the frequency distributions of CI-Base : CI-New are presented, where CI-Base and CI-New are the CI of the base and new tasks, respectively; “H” denotes the harmonic mean [57] of base-task and new-task accuracies.

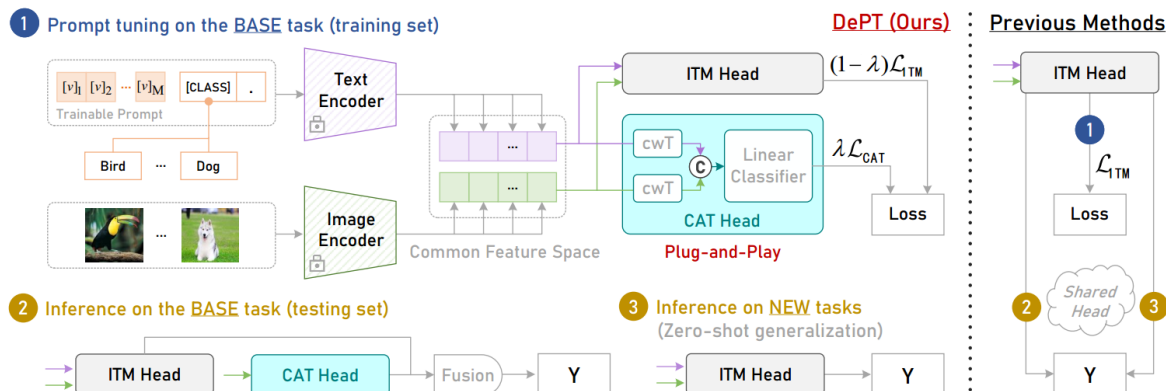


Figure 2. Illustration of our DePT framework (in a CoOp [58] style). Unlike previous methods (right) that use the same Image Text Matching (ITM) head for training/inference on the base task and zero-shot generalization on new tasks, our DePT (left) employs a Channel Adjusted Transfer (CAT) head to capture *base-specific* knowledge in an isolated feature space, so as to maximally preserve *task-shared* knowledge in the original feature space for improving zero-shot generalization on new tasks. At inference, we further boost the performance on the base task by simply fusing base-specific and task-shared knowledge obtained by the two heads. © denotes the concatenation operation.

从头到尾只有一组可学习的prompt，设计了两个独立的分类头，构建一个隔离的空间（CAT Head）来吸收基类知识，保护原始空间（ITM）不被破坏。

CAT：领域特定知识（更关注增强基类性能）

ITM：通用知识

所以基类评测用混合，新类评测尽量用ITM。

对于可更新的参数：CAT损失函数更新CAT Head内部参数，总损失函数更新提示词。

这么看来，ITM侧相比单一的CoOp会非常温和（因为专业微调都是CAT去做了，但是提示词更新又受到ITM的影响），所以新类性能增加？

我们的做法是，微调对齐原始clip侧（通用知识）。

可更新的参数（Learnable Parameters）只有两类： $\vartheta+1$

1. **Trainable Prompt (P)**: 输入在文本端的连续提示词向量 $[v]_1[v]_2...[v]_l$ 。 ϑ

2. **CAT Head 内部的参数**:

- 缩放向量 γ 和平移向量 β （属于 cwT 层）。 ϑ
- 投影矩阵 W （属于线性分类器）。 ϑ

参数更新的时机与梯度流向：

更新发生在微调训练的每次迭代（Iteration/Step）中。模型会同时计算两个 Head 的损失，然后按比例相加得到总损失：

$$\mathcal{L} = \lambda \mathcal{L}_{CAT} + (1 - \lambda) \mathcal{L}_{ITM}$$

2.DPC (CVPR2025)

作为一种即插即用方法，性能提升比DePT更多。

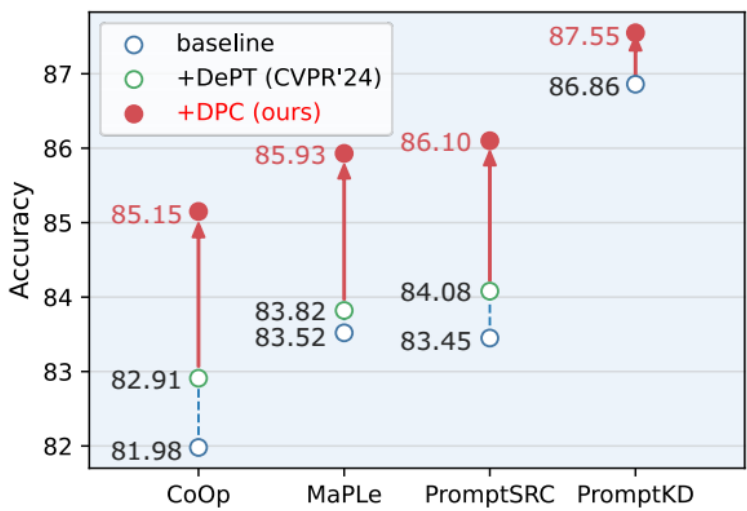


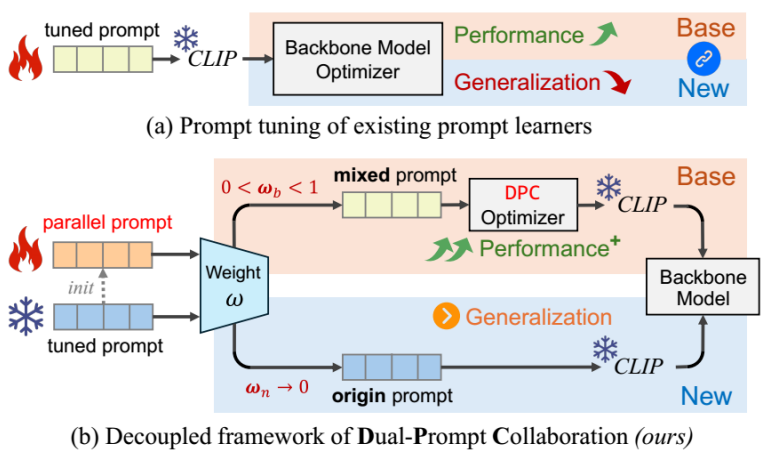
Figure 1. Average classification accuracy of 4 mainstream prompt tuning backbone models on base (target) classes over 11 datasets. DPC achieves state-of-the-art performance compared with base-lines and another leading plug-and-play model DePT [45].

困难点主要在于：模型微调过分聚焦于基类知识，过拟合，遗忘了通用知识，在新类上泛化性很差。

（但是现在更多关注于将基类和新类解耦，基类尽量影响不到新类，从而让新类性能不下降，这是首要目标，其次因为解耦，基类微调往往会增强，基类性能上升，**但是并没有考虑新类性能的显著提升**）

现有方法主要做法：损失函数设计（一致性约束（特征，分布层面），梯度）；提示词限制；引入新的提取器；引入外部知识（大模型生成的或者百科知识）。

该方法主要考虑：基类和新类优化目标是相互冲突的，应该分开优化。



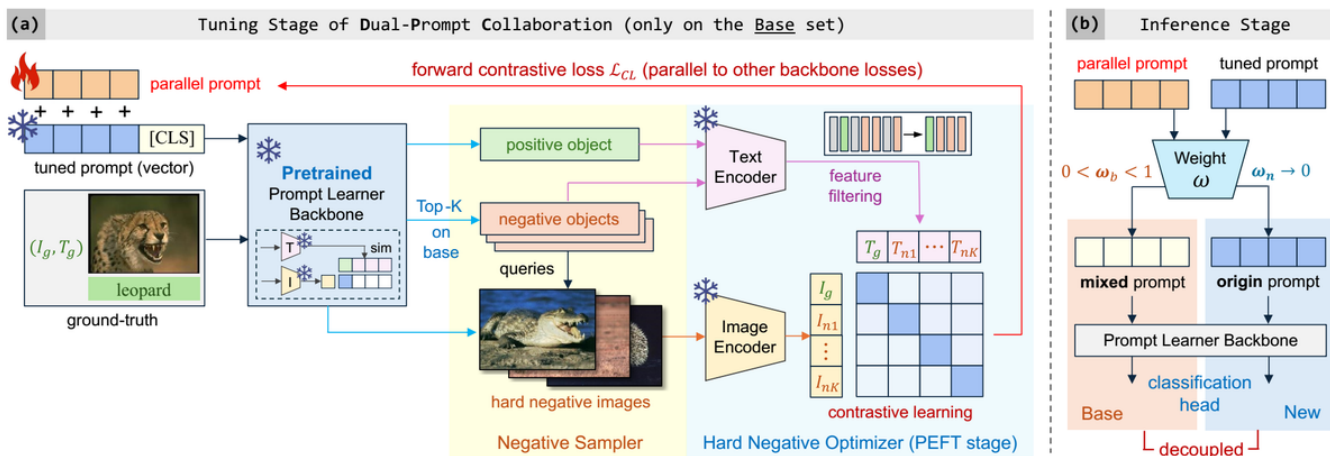


Figure 3. Overview of our proposed DPC. In (a) fine-tuning stage, DPC initializes parallel prompt P' based on tuned prompt P obtained by fine-tuning backbone. Negative Sampler applies tuned prompt P as query to sample hard negatives, then feed them into HNO optimizer to enhance base tasks. In (b) inference stage, DPC decouples base and new tasks by independent weight accumulation on dual prompts.

总体架构可以这么理解：

- 1.首先延续传统工作例如CoOp中的提示词prompt设计，初始化原始P，用P微调基类，得到微调后的提示词P；
(当然，鉴于一致性对比，时间和算力消耗，这部分可以用CoOp中一半的epoch来做)；
- 2.将P完整地复制一份，得到P'，用于基类加强的微调学习；
- 3.具体来说，基类加强微调的时候，**首先利用原始的P再次进行微调**，得到了分类结果中难以区分类别的难样本对（**topk**，其中有一个是正确分类的类别的图片和文本信息，以及k-1个相似度最高的但是并不是正确分类的图像文本对，用P是因为P已经微调更新过，再次使用准确率高），**归一化**，将这k个图像文本对构建新的微调输入批次，再来进行一次微调，并且这次微调设计了更加强的**损失函数**（用CoOp中一半的epoch来做）；
- 4.上述基类加强微调只更新P'。

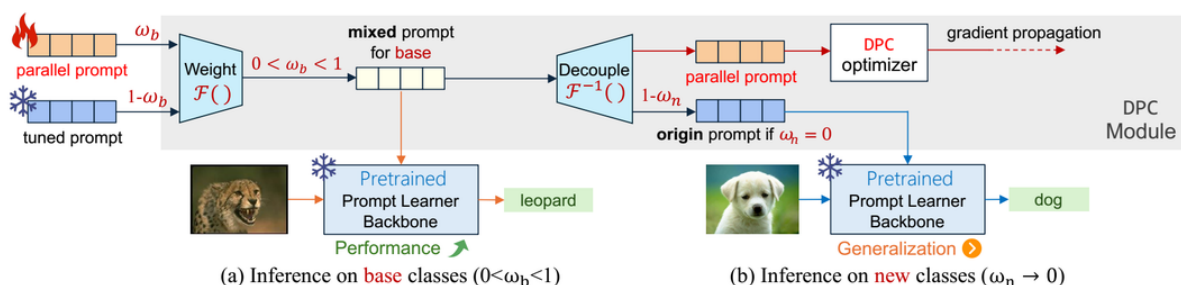


Figure 4. Weighting-Decoupling structure in DPC. This structure allows DPC to continuously optimize the parallel prompt P' during the tuning phase and to endow separate accumulated weights to dual prompts (P and P') during (a) inference stage on base classes and (b) inference stage on new classes.

- 5.在基类上进行评测的时候，使用的是**混合prompt**，基类加强微调后的prompt。
- 6.在新类上，直接做测试，利用的是在基类加强微调混合过后并且解耦的提示词P和P'，进行线性加权。但是倾向于**只利用最开始的提示词P**，P'的加权因子无限趋近于0。

是指在new类测试时，手动调整加权，P'的权值趋近于0；base类测试再换一个好的权值？这不算作弊吗（bushi

P': 领域特定知识（更关注增强基类性能）

P: 相对于P'来说是通用知识，但其实也是微调过后的领域特定知识

所以基类评测用混合，新类评测尽量用P。

代码块

```
1  # T: mini-batch of text annotations
2  # I: mini-batch of images
3  # W: collaboration weight ( $W_b$  = base class,  $W_n$  = new class)
4  # HNS: Hard Negative Sampler
5  # FF: Feature Filter
6  # CE: Cross Entropy loss
7
8  ### Dual prompts initialization
9  tuned_prompt = backbone_prompt_learner(T, I) # frozen
10 parallel_prompt = nn.Parameter(tuned_prompt) # learnable
11 mixed_prompt =  $W_b$  * parallel_prompt +  $(1-W_b)$  * tuned_prompt
12
13 ### STAGE 1: Training stage on base class
14 # obtain features of image and text
15 T_feat = text_encoder(parallel_prompt) # [n_cls, dim]
16 I_feat = image_encoder(I) # [batch, dim]
17
18 # apply negative sampler to get hard negative features
19 # size of T_feat and I_feat after sampling: both are [batch*TopK, dim]
20 for (text, image) in batch:
21     hn_t, hn_i = HNS ((text, image), tuned_prompt) # [TopK - 1]
22     text_feat = FF (text_feat, [text_id, hn_t_id]) # [TopK, dim]
23     img_feat = torch.cat([img_feat, image_encoder(hn_i)]) # [TopK, dim]
24
25 # Hard Negative Optimizer
26 logits_img = logit_scale * hn_I_feat @ filtered_T_feat.t()
27 logits_txt = logits_img.t() # [batch*TopK, TopK*batch]
28 ids = torch.arange(batch*TopK)
29 loss = (CE(logits_img, ids) + CE(logits_txt, ids)) / 2
30
31 ### STAGE 2: Inference on base & new class
32 # inference on base
33 logit = similarity_head(text_encoder(mixed_prompt), I_feat)
34 # inference on new
35 mixed_prompt =  $W_n$  * parallel_prompt +  $(1-W_n)$  * tuned_prompt
36 logit = similarity_head(text_encoder(mixed_prompt), I_feat)
```

自然的想法是：

1.增强基类的同时能否增强新类？P'确实更拟合于基类和特定知识，但是也可以考虑让它不偏离通用知识。 直接对P'进行梯度正则？基类加强微调时，每次更新P' 的时候，要求对齐P。（但是P也是微调后的，应该对齐原始CLIP吧？）

2.拆分？拆分什么？把提示词按照粗粒度和细粒度来拆分？提示词的粗粒度/细粒度，以及提示词有时也能代表通用/专业知识，这和两个因素有关：

- i)提示词本身的设计（ATprompt（ICCV2025，加入静态属性），AnchorOPT（动态属性））
- ii)提示词在深层神经网络中的位置（有研究表明浅层管通用，深层管专业，MMA（CVPR2024）（但是尽量不要引入adapter），MMRL（CVPR2025），MMRL++（IJCV2026），HicroPL（ICCV2025））

讨论后新增加的想法：

3.结合DePT和DPC DPC可以保证基类性能有提升，新类性能不掉，DePT可以保证新类性能提升，可以做一个针对于新类性能的prompt P''，再有机结合一下，也可以造出来一个新类微调的过程（三阶段？）

梯度正则和对齐再看。

4.重点关注hard样本选择的问题，看看到底怎么实现的。需要关注以下几点：

- i)本身选出来的东西是否真的有必要？（消融实验看上去很必要）

有些分类正确的，不需要再选择hard样本。是否置信度高的一定是正确类别？也有可能Topk不见得能保证分类正确性，也许Top1和Top2很接近，我只要重点关注两个就行。

- ii)我只要保证改错比改对少就行。还有一个笨方法，把每个的预测分类以及topk取出来，和真实标签对比。

5.引入伪标签并不算作弊，task不一样，KD task也不一样。只要图像-文本对不要都看就行，看一个没事，伪标签又不一定是正确的。新类能不能引入微调，再看。

6.直接用P' 做基类预测应该也行，wb只是0.2，P'动机不是太强。

3.MAO（ICME2025）

直接对新类下手，增强新类性能。

模型结构不变，重点关注**数据增强与损失函数设计**。

这个是喂了new类数据？还是说仅喂标签或者图片，没“new类”

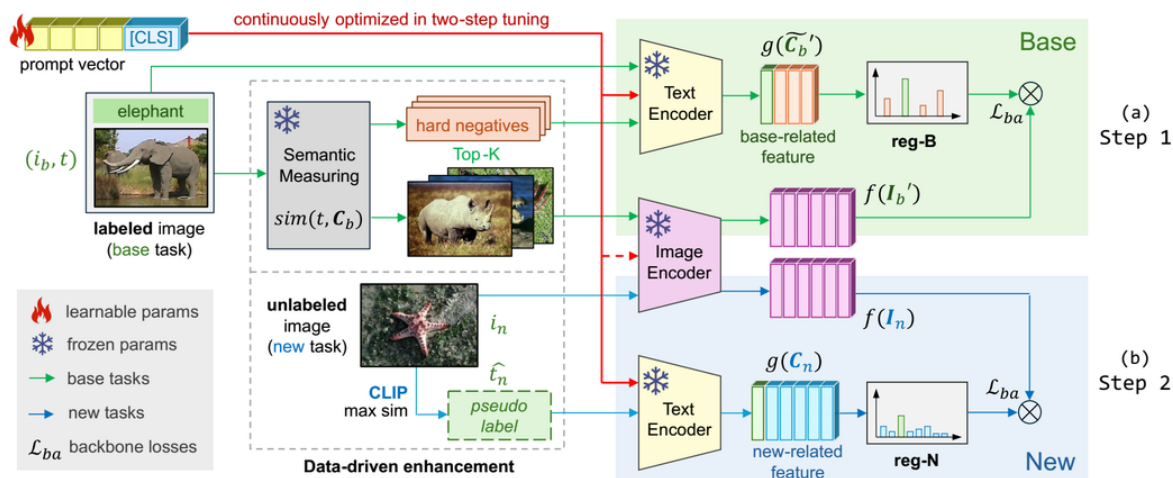


Figure 2. Framework of proposed MAO. MAO builds a **two-step** fine-tuning structure without altering components of prompt tuning backbones. In (a) **base** tasks, MAO introduces a hard negative sampler as Data-Driven Enhancement (DDE), and an Alterable Regularization (reg-B) that guides the model in learning the feature distribution of hard negatives and keeps generalization. Then in (b) **new** tasks, rapid pseudo-labeling is performed on unlabeled images as DDE using shared-parameter CLIP, followed by reg-N to constrain the fine-tuning on new classes. The inference process follows the settings of the original backbones.

基类只训练困难样本；（前半段epoch）

对于新类别中的一张无标签图片，喂进去得到图像特征。用原始CLIP直接对这张图片进行预测，选择TOP1概率的类别作为伪标签，将文本信息喂到文本编码器微调。（后半段epoch）

Transductive ZSL? ? Test-Time Adaptation? ? ? 数据泄露? ? 感觉偏题了

4.PromptKD (CVPR2024)

Transductive ZSL 知识蒸馏

蒸馏也挺作弊的吧。。。。

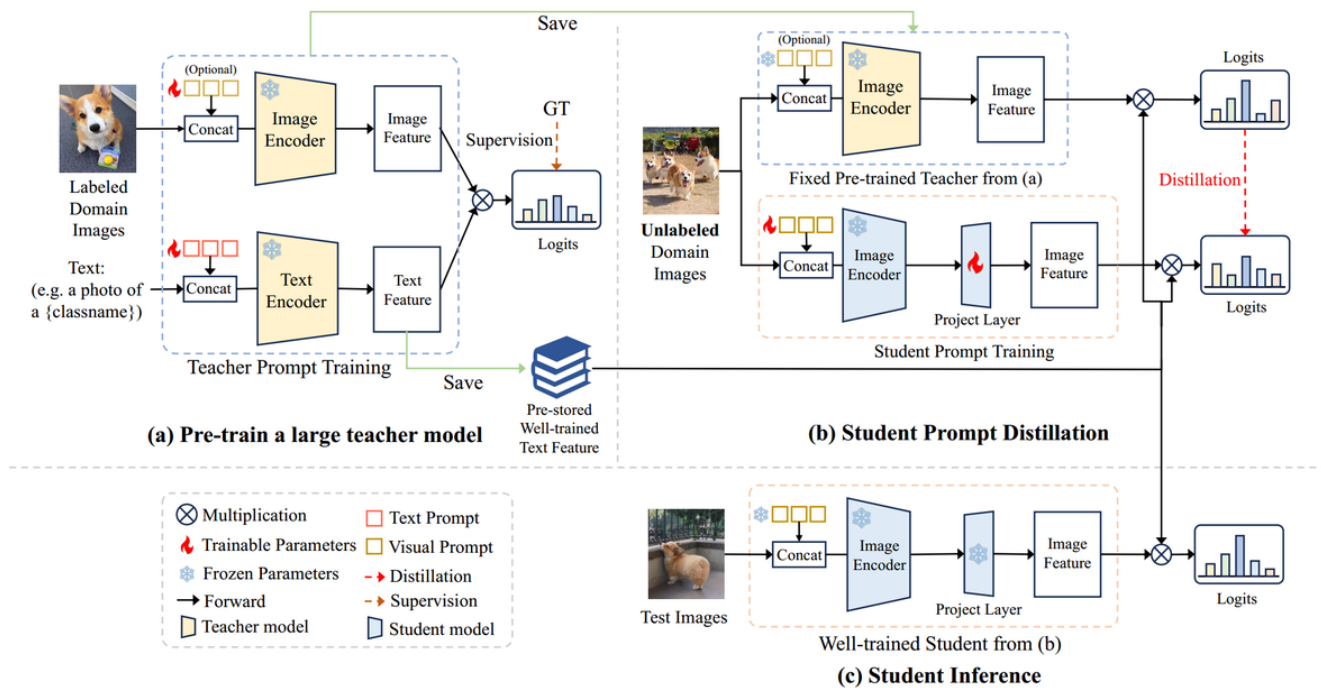


Figure 3. An overview of our proposed prompt distillation (PromptKD) framework. (a) We first pre-train a large CLIP teacher model using existing state-of-the-art prompt learning methods with labeled training images. Then we save the well-trained text features of all possible classes for the next stages. (b) During the distillation stage, the training is focused on student image prompts and the project layer, and there are no extra computational expenses associated with the text encoding process when utilizing the pre-saved text features as class vectors. (c) Finally, the well-trained student and pre-stored class vectors are utilized for inference.

为什么要用知识蒸馏？内部知识蒸馏？原始CLIP模型。

知识蒸馏可以大幅度节省模型参数量，内存，运行时间，也可以让小模型学习大模型的基类特异能力。

1.PromptKD首先微调大模型，将大模型的文本特征保留（基类微调）。

2.同时喂给大模型和小模型无标签的图片（我觉得可以理解为新类的图片），和大模型保存下来的文本特征做匹配，并且用project将小模型视觉特征维度映射到和大模型文本特征可以匹配的维度，之后让小模型对齐大模型。只更新prompt和project。

3.用小模型做测试。

代码块

```
1 # tea_t: text encoder of teacher CLIP
2 # tea_i: image encoder of teacher CLIP
3 # stu_i: image encoder of student CLIP
4 # l_tea: teacher output logits
5 # l_stu: student output logits
6 # Proj: Feature Projector
7
8 # init
9 f_txt_t = tea_t(txt_of_all_classes)
10
11 # forward
12 for img in unlabeled_dataset:
13     f_img_t = tea_i(img)
```



```

14     f_img_s = stu_i(img)
15     f_img_s = Proj(f_img_s)
16
17     # get output predictions
18     l_tea = f_img_t * f_txt_t.t()
19     l_stu = f_img_s * f_txt_t.t()
20
21     # calculate distillation loss
22     loss = KLDivergence(l_stu, l_tea)
23     loss.backward()

```

辅助参考论文

1.ATprompt

详细解读可参考知乎文档。

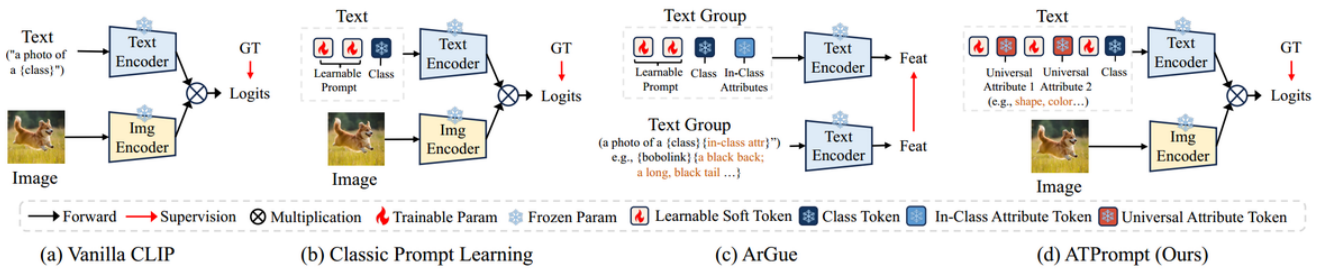


Figure 2. Architectural comparison among existing methods. (a) Vanilla CLIP employs a hand-crafted text template as input to the text encoder. (b) Classical prompt learning proposes a new text form that concatenates multiple learnable soft tokens with class tokens. (c) ArGue [49] employs multiple *in-class* attributes mined by LLMs as supplementary information. These attributes are utilized to construct distinct text groups, which serve as *learning targets*, thereby regularizing the learning of soft tokens. The final prediction is achieved by ensembling all groups. (d) Our ATPrompt treats *universal attributes* as *learning components* and anchors them into existing soft prompt templates. Though this operation, we expand the learning space of soft tokens to multi-dimensional attribute levels and facilitate the alignment of images with unknown class texts.

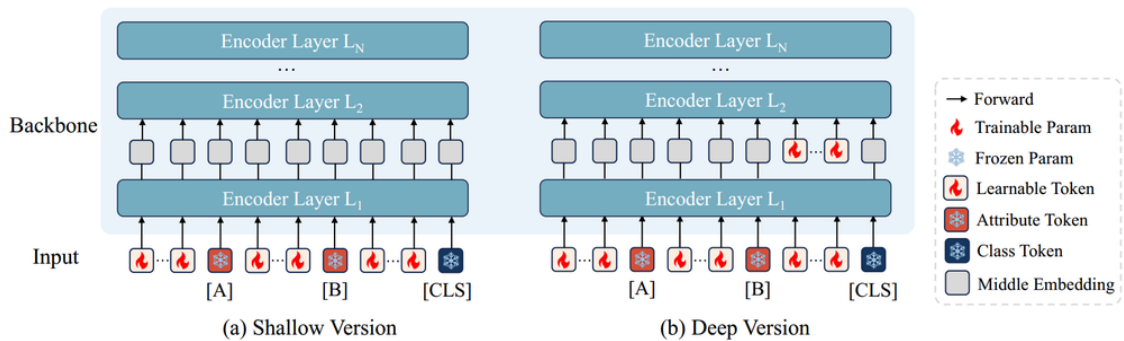
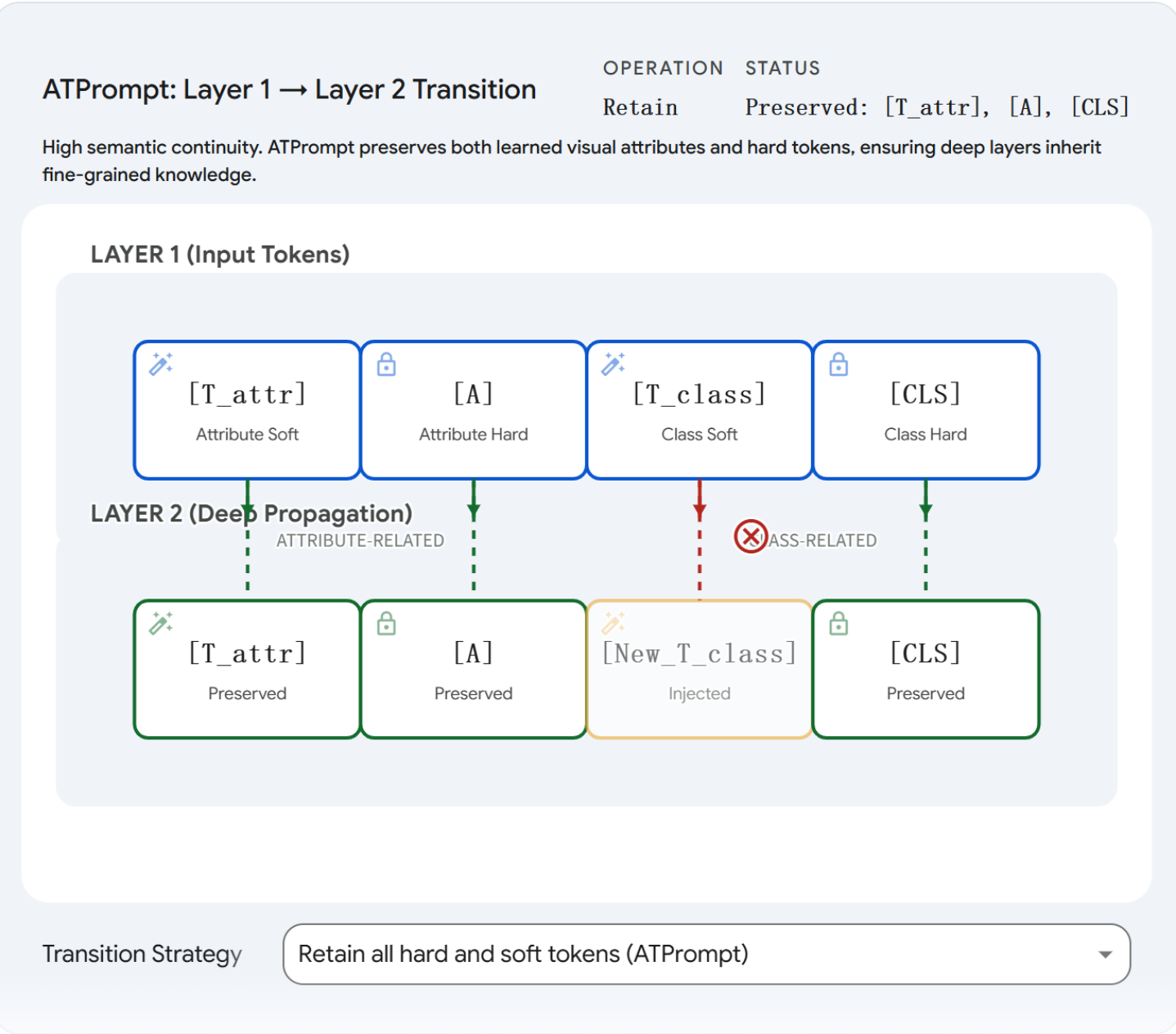


Figure 3. An illustration of the computation process for shallow and deep versions. Take two attributes [A] and [B] as examples. (a) The shallow version concatenates hard attribute tokens, soft prompt tokens, and class tokens and inputs them into the encoder for calculation. (b) The deep version uses the same input but discards the class-related soft prompt tokens after calculating the self-attention and introduces them again before the next layer. These two forms can be compatible with existing methods of varying prompt depths, including input-level ones like CoOp [64], CoCoOp [63] and depth-level ones like MaPLe [19].

第二层就是深层，深层和浅层的提示词构建区别如下：



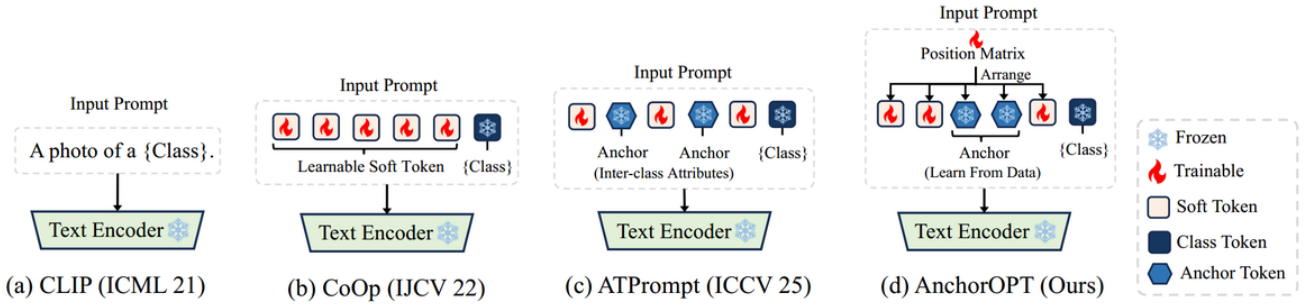


Figure 1. Comparison of fundamental input prompt structures. (a) CLIP[32] employs manually designed text templates. (b) CoOp[48] introduced prompt learning for CLIP adaptation using learnable soft tokens concatenated with fixed category tokens. (c) ATPrompt [25] incorporates explicit, fixed attribute tokens (e.g., “color”, “shape”) to guide soft token learning via an attribute-based template. (d) AnchorOPT utilizes implicit anchors learned from data to guide the learning of soft tokens and proposes a learnable position matrix that dynamically adjusts the prompt sequence according to downstream requirements.

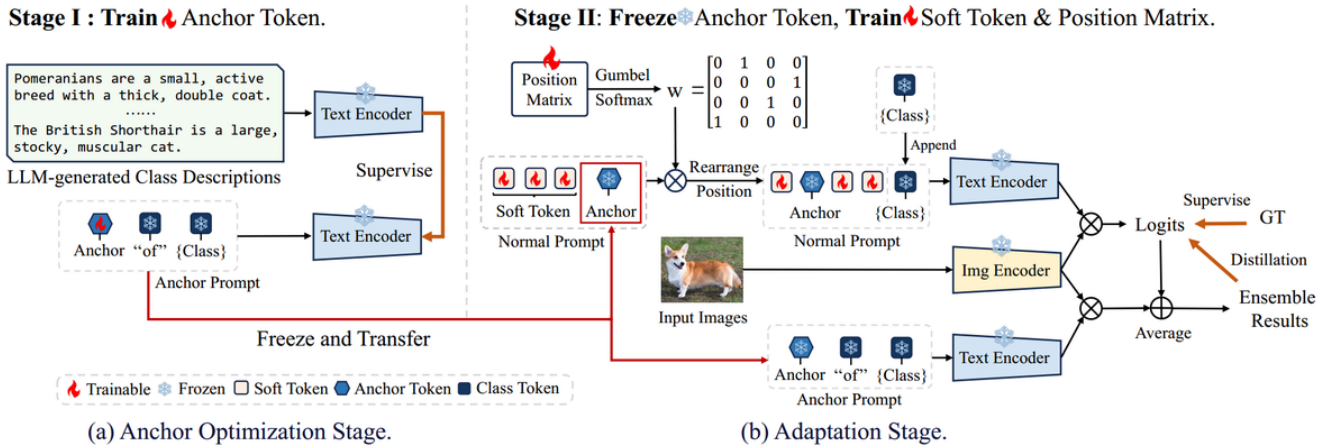


Figure 2. The dynamic anchor token training process comprises two stages: (a) Anchor Optimization: Anchor tokens are initialized as learnable parameters and optimized using LLM-generated category descriptions. The resulting anchors are frozen for the subsequent stage. (b) Adaptation: Soft prompts and the position matrix are jointly optimized for downstream tasks, with knowledge distillation from ensemble results providing auxiliary supervision.

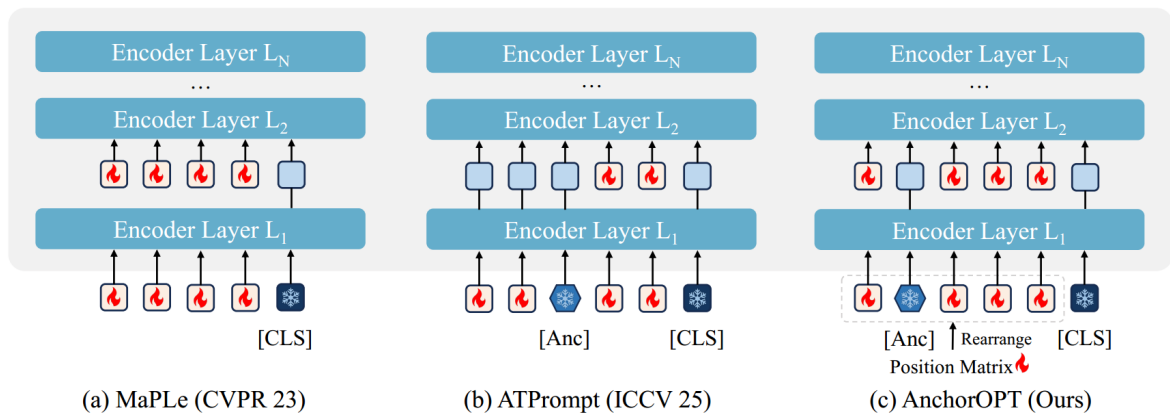


Figure 3. Computational process in deep prompt learning variants: (a) MaPLe drops and reintroduces all soft tokens after each Transformer block. (b) ATPrompt retains all attribute-related hard/soft tokens while discarding class-related soft tokens. (c) AnchorOPT dynamically reorders tokens via the position matrix, retaining only anchor tokens and discarding all soft tokens during processing.

延续到深层：

ATprompt属性词元要一脉相承，包括浅层可以学习的（不更新了）+锚点，而类别可学习词元每层都重新生成；

AnchorOPT则只传播锚点，浅层属性可学习soft token和类别可学习soft token在深层一起重新生成并更新。

3.HicroPL

浅层网络包含通用知识，深层网络包含专业特异性知识。浅层粗粒度，深层细粒度。

浅层：文本特征已经成熟，引导视觉特征。浅层代表类别。

深层：视觉特征已经成熟，文本特征逐渐衰退，视觉引导文本。深层代表实例。

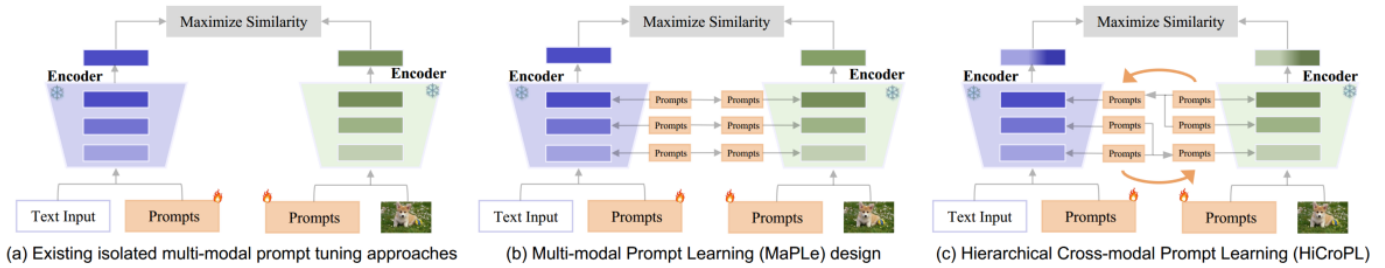


Figure 1. Comparison of HiCroPL with existing prompting approaches. (a) Most existing methods adopt uni-modal adaptation or isolated multi-modal solutions to fine-tune CLIP. (b) Multi-modal Prompt Learning (MaPLE) proposes a *one way* (i.e. text-to-vision) coupling function to bridge the two modalities, but visual concepts lack pathways to guide textual semantics. (c) HiCroPL introduces a bidirectional knowledge flow mechanism between the two modalities, enabling them to refine their semantics mutually for deep alignment. Besides, the representation used for downstream decisions contains rich intermediate features for improved generalization.

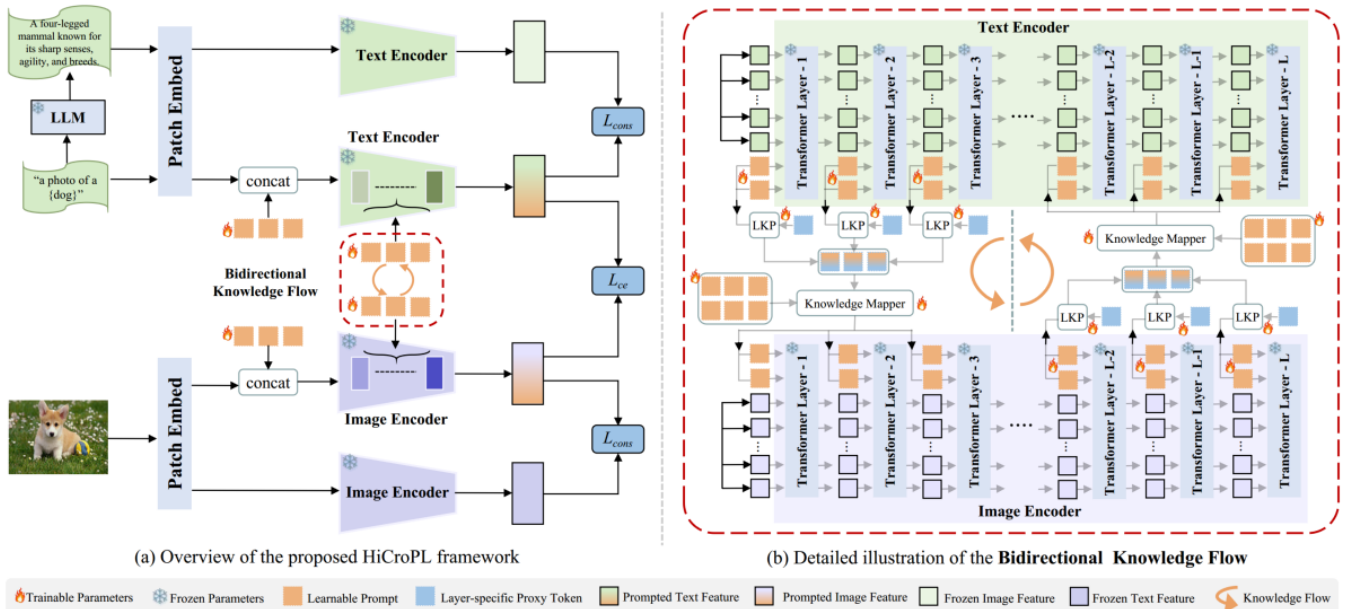


Figure 2. (a) Overview of the proposed HiCroPL framework. (b) Detailed illustration of the Bidirectional Knowledge flow mechanism. From Layer 1 to k , the LKP first initializes layer-specific proxy tokens to encapsulate the key information relevant to the current layer, which then guide visual prompt refinement via the mapper $\mathcal{M}$. The reverse flow from Layer $k+1$ to L follows an identical process.

4.MMRL

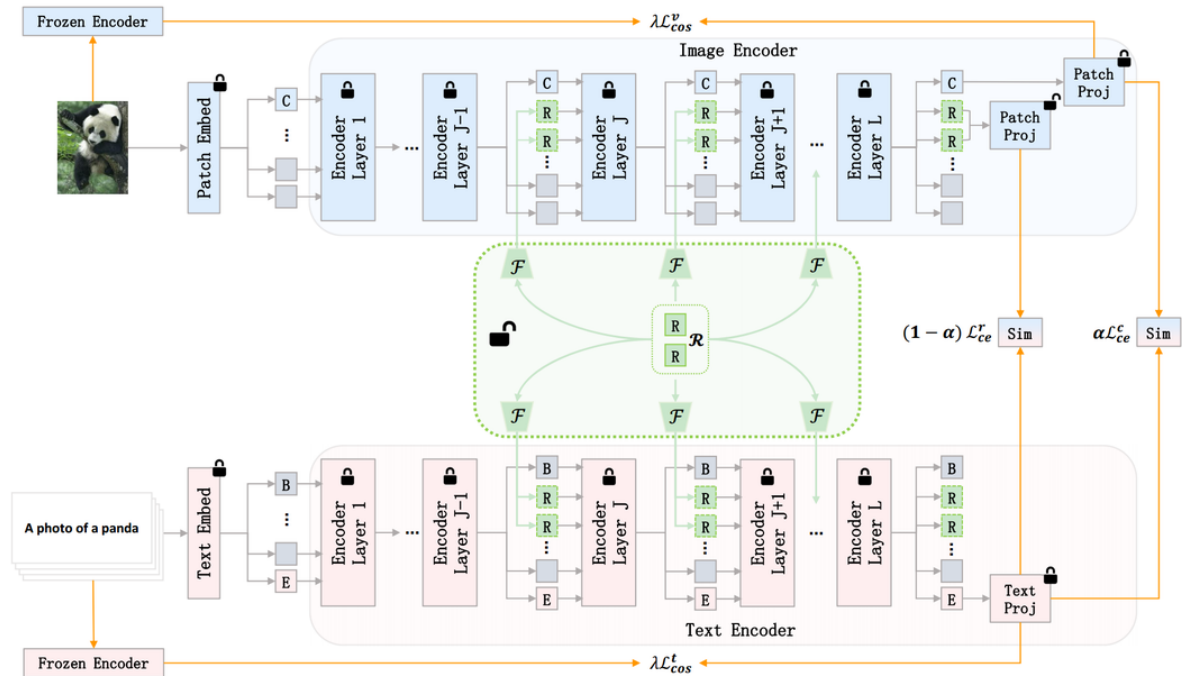


Figure 2. MMRL training framework. Here, ‘C’ denotes the class token, ‘B’ the BOT token, ‘E’ the EOT token, $\mathcal{R}$ our representation space, and ‘R’ the representation token. **Only the representation space $\mathcal{R}$, mapping function $\mathcal{F}$, and the patch projection layer for the representation tokens are optimized**, while the entire pre-trained CLIP model remains frozen. To preserve generalization knowledge, we integrate representation tokens in both encoders starting from layer J .

浅层代表通用知识（类别层面），深层代表特定知识（实例层面）。

抛弃CoOp类型的类别soft tokens，在浅层直接采用原始的CLIP。

在深层引入共享空间R，高斯初始化R，经过F，映射到神经网络中，表征soft tokens。

最后损失函数：类别ce+表征ce+视觉侧和语言侧分别与ZSCLIP对齐。

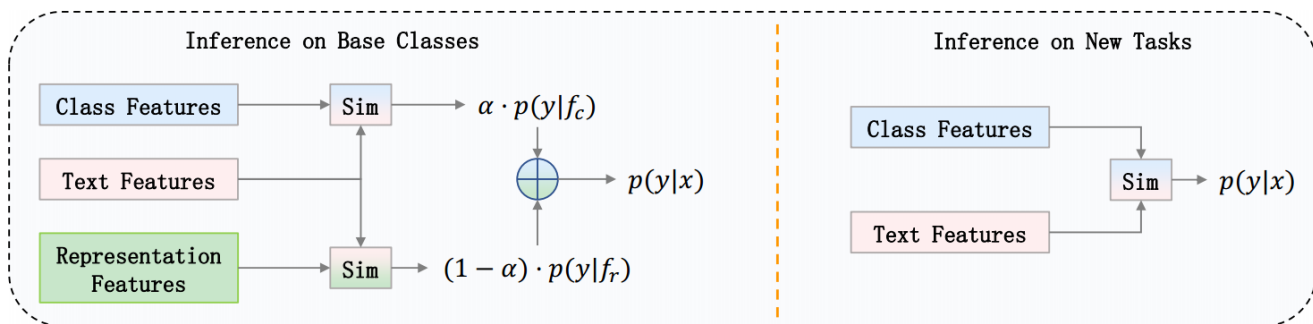


Figure 3. MMRL inference process, where different tasks utilize distinct features.

推理时面对基类和新类分而治之。

5.MMRL++

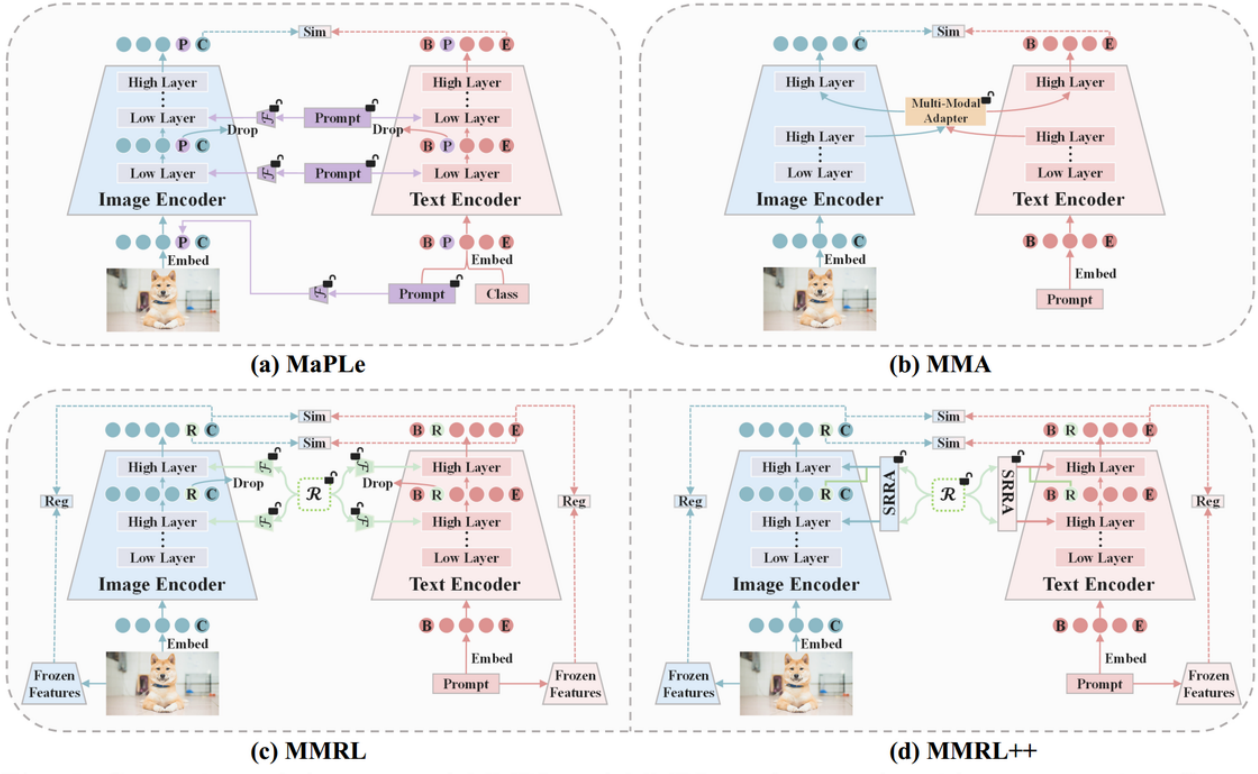


Fig. 3: Comparison of the proposed MMRL and MMRL++ frameworks with representative efficient multimodal transfer learning approaches—MaPLE from prompt learning and MMA from adapter-style learning.

1. 第一板斧：共享-残差表示对齐器 (SRRA) —— 解决参数冗余

MMRL++ 彻底废弃了每一层独立的映射器。它巧妙地借用了 LoRA（低秩微调）的思想，将每一层的权重矩阵 W_i^m 拆解为公式 (27)：

$$W_i^m = W_{shared}^m + A_i^m B_i^m$$

- W_{shared}^m ：这是所有层**全局共享**的基础权重矩阵。既然是共享的，不同层计算的梯度就可以汇聚到这里，实现了跨层的隐式梯度交流。
- $A_i^m B_i^m$ ：这是针对特定层的残差知识，通过低秩分解（Low-rank decomposition，比如秩 $r_1 = 4$ ）实现。
- **结果**：通过这一招，MMRL++ 的可学习参数从 4.992M 瞬间暴降至 0.813M（砍掉了 84%），同时还能维持甚至提升性能。

2. 第二板斧：渐进式表示组合 (PRC) —— 解决层间孤立

既然 Token 是拼接到网络里的，那上一层处理完的 Token，怎么传给下一层？

- **MMRL 的做法**：直接把上一层提取好的 Token 丢弃，下一层重新用映射器生成新的 Token 塞进去（层与层之间特征断层）。
- **MMRL++ 的做法 (PRC)**：将“当前层刚生成的 Token (R)”和“上一层传递下来的历史 Token (O)”进行加权融合。以视觉端公式 (31) 为例：

$$[c_i, O_i^v, E_i] = \mathcal{V}_i([c_{i-1}, \beta R_{i-1}^v + (1 - \beta) O_{i-1}^v, E_{i-1}])$$

- **结果**：这个公式让模型像滚雪球一样，逐层积累针对具体图像（Instance-level）的上下文细节，显著提升了训练稳定性和泛化能力。

6.MMA 纯adapter方法

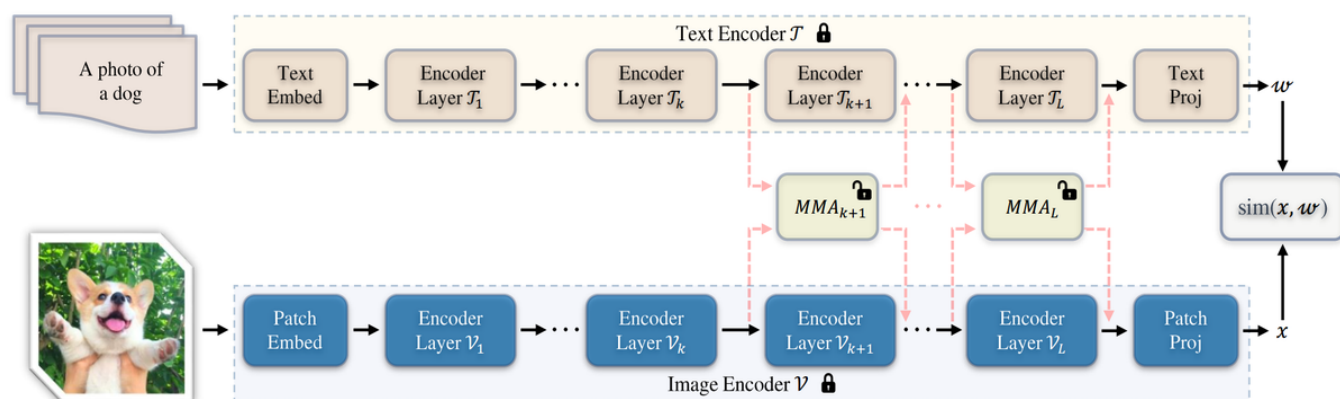


Figure 2. The proposed Multi-Modal Adapter (MMA) for the transformer-based CLIP models. Our MMA tunes both image and text encoders. **Only the extra adapters are optimized**, while the whole pre-trained CLIP models are frozen. **In our method, only a few higher layers ($\geq k$) of each encoder will be tuned based on our analysis to strike a good balance between discrimination and generalization dilemma.** Moreover, our MMA shares weights between image and text representations to learn shared cues from different branches. By this design, our MMA eliminates feature-wise interactions between each image-text pair [85], greatly reducing the computational cost.

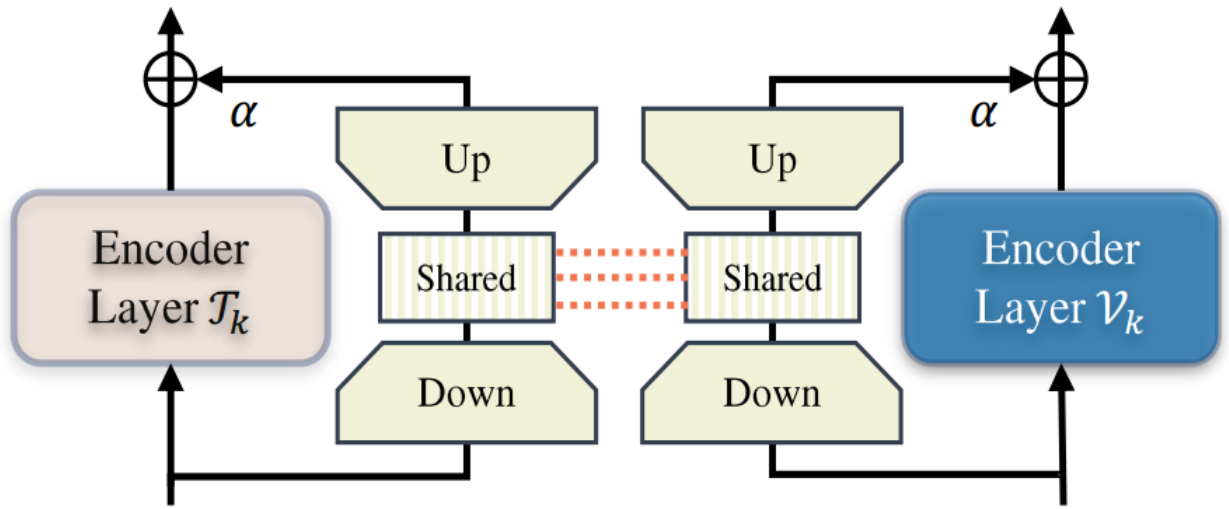


Figure 3. The newly designed multi-modal unit. It contains separate projection layers (“Down” and “Up”) to tune different modals’ encoders, as well as a shared projection layer (“Shared”) to build a strong connection between vision and language branches.

一、宏观拦截：旁路残差连接 (Residual Bypass)

假设现在网络运行到了第 k 层（注入层），输入进来的视觉特征张量是 z_k 。

如果没有 Adapter（原始 CLIP），数据直接穿过这一层的 Attention 和 FFN，去往下一层。

如果有 MMA Adapter，数据在主干路上走的同时，会被复制一份，送入我们架设的旁路（Adapter $\mathcal{A}$ ）。最后，旁路的输出会乘以一个缩放因子 α ，再加回主干路上。

用物理公式表达就是：

$$Output = \text{LayerNorm}(z_k + \alpha \cdot \mathcal{A}_k^v(z_k))$$

物理直觉： Adapter 就像是一个外挂的音效均衡器。主声音（ z_k ）照常播放，均衡器（ $\mathcal{A}_k^v$ ）提取出特定的高低音（任务特异性特征）后，按照 α 的音量大小，混音回主声道里。

二、微观沙漏：Adapter 的内部矩阵乘法

这个旁路均衡器 $\mathcal{A}_k^v$ 里面到底是什么代码？这正是 MMA 全篇最精妙的设计，它被称为***“跨模态沙漏结构 (Cross-modal Bottleneck)”***。

我们将视觉适配器 $\mathcal{A}_k^v$ 和文本适配器 $\mathcal{A}_k^t$ 放在一起对比看。在代码里，它们各自要经历 3 次矩阵乘法：

1. 独立降维 (Down-Projection)

特征刚进入旁路时，维度很高（例如 $d = 768$ ）。为了减少参数量并提纯特征，首先用各自独立的线性层（Linear Layer）将它们压缩到一个极低的维度（例如 $r = 64$ ）。

- 视觉端： $h_v = \delta(W_{kd}^v \cdot z_v)$
 - 文本端： $h_t = \delta(W_{kd}^t \cdot z_t)$
- * (δ 是 ReLU 或 GELU 激活函数，引入非线性计算)。

2. 共享桥梁 (Shared-Projection) —— 灵魂所在！

接下来，特征来到了沙漏的最窄处。注意看，在这里，视觉和文本特征强行通过了同一个线性矩阵 W_{ks} ！

- 视觉端继续： $m_v = \delta(W_{ks} \cdot h_v)$
- 文本端继续： $m_t = \delta(W_{ks} \cdot h_t)$
- 物理直觉：** 为什么这是 MMA 的灵魂？因为在反向传播计算梯度（Gradient）时，视觉图片的 Loss 算出来的梯度会流经 W_{ks} 并更新它；文本标签的 Loss 也会流经 W_{ks} 并更新它。这强制逼迫原本孤立的视觉和文本参数，在这个低维度的房间里发生了激烈的物理碰撞和特征对齐。文本的泛化性通过这个矩阵流向了视觉分支！

3. 独立升维 (Up-Projection)

特征在这个共享矩阵里交流完毕后，必须回到原来的高维度（ $d = 768$ ），才能与主干路的残差相加。此时再用各自独立的线性层将维度放大。

- 视觉端输出： $\mathcal{A}_k^v(z_v) = W_{ku}^v \cdot m_v$
- 文本端输出： $\mathcal{A}_k^t(z_t) = W_{ku}^t \cdot m_t$

